

Table of contents

- [HUML specification v0.1.0 Experimental](#)
 - [Encoding and basic structure](#)
 - [Data types](#)
 - [Indentation](#)
 - [Spaces](#)
 - [Comments](#)
 - [Keys and values](#)
 - [Scalars](#)
 - [Strings](#)
 - [Single-line strings](#)
 - [Multi-line strings](#)
 - [Vectors](#)
 - [Lists](#)
 - [Dicts](#)
- [Example](#)
 - [Why ::?](#)
 - [Document root](#)
 - [Kitchensink example](#)



[Home Specification Code Playground](#)

HUML specification v0.1.0 Experimental

HUML is a machine-readable markup language with a focus on readability by humans. It borrows YAML's visual appearance, but avoids its complexities, ambiguities, and dangerous footguns. It is very strict about indentation and spaces specifically to ensure consistent form across contexts for readability.

HUML is tailored for configuration, documents, and datasets.

Encoding and basic structure

- UTF-8 encoding.
- Line breaks: Unix-style (`\n`).
- Blank lines are ignored.
- Optional version directive at the top: `%HUML <version>` (e.g., `%HUML v0.1.0`). If this is not present, a parser is to apply the latest version of the specification.

Data types

HUML supports scalar (string, number, bool, null) and vector (list, dict) data types.

Type	Description	Example
String	Only canonical (decoded) representation of Unicode strings. No <code>\u</code> or <code>\U</code> or any other escaped or encoded representations.	Allowed: "Hello 🌍" Not allowed: "Hello \u1F30F"
	Single-line string: Always quoted with double quotes. " and \ must be escaped.	"foo", "Hello World", "Hi \\"all\\" ", "Back\\Slash"
	Multi-line string: Wrapped in <code>```</code> (three backticks) to preserve spaces or <code>"""</code> (three double quotes) to strip them.	
	Characters inside a multi-line string do not need to be escaped. Escape sequences with backslashes like <code>\n</code> are treated literally.	
Number Integer		123, +123, -123
	Float (64-bit, IEEE 754 double-precision floating-point)	3.14, -0.5
	Special values. The special numerical values are unquoted.	nan, inf, +inf, -inf
	<i>Notations</i>	
	Exponent , denoted by a lowercase <code>e</code>	1e10, 6.022e23
	Hex , begins with <code>0x</code>	0x1A, 0xCAFE
	Octal , begins with <code>0o</code>	0o12, 0o755
	Binary , begins with <code>0b</code>	0b1010, 0b11011001
	Numbers can have arbitrary underscore characters for readability, which are simply ignored while parsing.	1_00_0000
Boolean	True or false values.	true, false
Null	Null value.	null
List	Array of arbitrary data types.	1, 2, "three"
	Definition can be inline or multi-line.	
	Multi-line items are prefixed with a <code>-</code> , like a bullet point list.	
	<code>[]</code> is a special signifier used to denote an empty list, e.g., items:: [].	[]
Dict	Unordered map of key-value pairs. Keys are strings and values can be of arbitrary data types.	hello: "world", num: 123
	Definition can be inline or multi-line.	
	Duplicate keys inside a dict are not allowed.	
	<code>{}</code> is a special signifier used to denote an empty dict, e.g., items:: {}.	{}

Indentation

- Strictly 2 spaces per indentation level. For multi-line list and dict vector definitions, indentation represents hierarchy.

Spaces

The presence of space characters is strictly controlled.

- Trailing spaces are not allowed on any line, including empty lines and comment-only lines, except for content within multi-line strings, where they are treated as content.
- The comment marker `#` must be immediately followed by one space before the comment contents, e.g., `# Comment` and not `#Comment`.
- Only a single space is allowed after the indicators `:`, `::`, and `-` and before the subsequent value.
- For multi-line vectors, `::` must be immediately followed by a line break, unless it is a comment starting with `#`.
- In inline lists, commas must not have preceding spaces and must be followed by exactly one space, e.g., `1, 2, 3`.

Comments

- Lines beginning with `#` are comments. Comments can have preceding spaces, but not trailing spaces.
- After `#`, there must be at least one space before any other character.

Comment-only line

key: "value" # Inline comment with a lot of preceding spaces.

Keys and values

- Keys are case-sensitive Unicode strings.
 - If (alphanumeric-underscore-dash) `a-zA-Z0-9_-` and starting with an alphabet, the key does not need to be quoted.
e.g., `foo: 123`, `foo-bar: "yes"`, `foo1: true`
 - If any other characters, including spaces, are present, keys must be quoted.
e.g., `"1foo": 123`, `"foo bar": 123`, `"ഭൂമി": "Earth"`
- Scalar keys are denoted by the key followed by a single colon (`:`).
- Vector keys are denoted by the key followed by double colons (`::`).
- No spaces are allowed before the `:` or `::`, and they must be followed by exactly one space before the value.
- The root of the document does not need a `:` or `::` at the top as the entire document's type is inferred from the first value that is encountered in the document.

Scalars

Scalars (string, number, boolean, null) are denoted by a single colon (:).

```
key: "value"  
number: 123  
boolean: true  
type: null
```

Strings

Strings can only be represented in their canonical (decoded) form. That is, encoded representations such as `\u` and `\U` are not permitted to ensure consistency and readability.

Single-line strings

Single-line strings may include the following formatting control characters using C-style escape notation with backslashes.

Escape Description

<code>\b</code>	Backspace
<code>\f</code>	Form feed
<code>\n</code>	Newline (LF)
<code>\r</code>	Carriage return
<code>\t</code>	Horizontal tab
<code>\v</code>	Vertical tab

Multi-line strings

In multi-line string blocks, no escaping is necessary. Characters are treated literally. As the beginning and ending markers can be derived from the indentation, there is no need to escape the three double quotes or backticks themselves in the string content.

Preserve spaces: ```

The content block must be indented by one level (2 spaces) relative to the key. These initial 2 spaces on each line is the minimum required indentation and is not considered as spaces. All other preceding and all trailing spaces are preserved as content. The opening indicator ``` must be after the indicator : separated by exactly one space. The closing indicator must be at the indentation level of the key.

```
description: ```  
  Line 1  
  Line 2  
  Line 3  
    All spaces are preserved.  
```
```

is parsed as:

```
Line 1
Line 2
Line 3
 All spaces are preserved.
```

### Strip spaces: ""

The content block must be indented by one level (2 spaces) relative to the key. All leading and trailing whitespace on each line of content is stripped. The opening indicator "" must be after the indicator : separated by exactly one space. The closing indicator must be at the indentation level of the key.

```
description: ""
 Line 1
 Line 2
 Line 3
 All spaces are ignored.
""
```

is parsed as:

```
Line 1
Line 2
Line 3
All spaces are ignored.
```

---

## Vectors

Vectors are denoted by double colons (::) and can be lists (arrays) or dicts (unordered key-value maps or dictionaries).

### Lists

- **Inline:** Comma-separated without trailing commas. Inline list items can only be scalar values and cannot contain vectors or nesting. They are meant to be a readability shortcut for small lists.
- **Multi-line:** A hyphen - denotes a list item, indented by 2 spaces relative to its parent. Multi-line lists can be nested and can contain vectors.
- **Empty list:** A vector can be marked as an empty list with the special value [].

```
inline_list:: 1, 2, "three"
```

```
multiline_list::
- 1
- 2
- "three"
```

```
[1, 2, "three", [1, 2, "three"]]
nested_list::
- 1
- 2
```

```

- "three"
- ::
 - 1
 - 2
 - "three"

[{"one": 1, "foo": "bar"}, {"two": 2, "foo": "baz"}]
list_of_dicts::
- ::
 one: 1
 foo: "bar"
- ::
 two: 2
 foo: "baz"

empty_list:: []

```

## Dicts

- **Inline:** Keys inside a dict are key: value pairs, comma-separated without trailing commas. Inline dicts can only contain scalar key-values and cannot contain vectors or nesting. They are meant to be a readability shortcut for small dictionaries.
- **Multi-line:** Keys inside a dict are key: value on a line indented by 2 spaces relative to its parent. Multi-line dicts can be nested and can contain vectors.
- **Empty dict:** A vector can be marked as an empty dict with the special value {}.

```

inline_dict:: one: 1, foo: "bar"

multiline_dict::
 one: 1
 foo: "bar"

{"one": 1, "foo": "bar", "nested": {"two": 2, "foo": "baz"}}
nested_dict::
 one: 1
 foo: "bar"
 nested::
 two: 2
 foo: "baz"

empty_dict:: {}

```

# Example

## Why ::?

1. The indicator :: immediately makes it apparent that what follows is a vector, to both human readers and parsers. Less guessing.

```

Inline list: [1, 2, 3, "four"]
foo:: 1, 2, 3, "four"

Inline dict: {"foo": {"bar": "baz", "one": 1}}
foo:: bar: "baz", one: 1

```

```
Multi-line list
foo::
 - 1
 - 2
 - 3
```

```
Multi-line dict
foo::
 bar: "baz"
 one: 1
```

2. It permits vectors to be defined inline without additional syntax such as `[ ... ]` or `{ ... }`. Opening and closing enclosures bring in complexities of keeping track of nesting and balancing closures, both for humans and parsers.

```
This is a list with one item: key = ["one"]
key:: "one"
```

```
Without :: it is not possible to represent an inline list
without an enclosure such as [...]. For example, key: ["one"]
Here, key = "one"
key: "one"
```

3. It avoids several ambiguities. In YAML, for example:

```
Although this looks like a list, it is a string: foo = "1, 2, 3"
foo: 1, 2, 3
```

```
Square brackets denote a list.
foo: [1, 2, 3]
```

```
Square brackets over multiple lines are also a list.
foo: [
1,
 2,
 3,
]
```

```
And this is also a list.
foo:
- 1
- 2
- 3
```

```
Although this looks like a nested list,
it actually translates to: {"foo": ["1 - 2 - 3"]}
WUT?
foo:
 - 1
 - 2
 - 3
```

## Document root

An empty HUML document (ignoring comments) is by default considered undefined. Since the root of the document does not have a key, an explicit indicator `:` or `::` is not required. The document's type is inferred dynamically from the first encounter of vector

or scalar values. Any subsequent type or value that appears throws a parser error, unless they are child elements of a vector.

Various scenarios are illustrated below.

The document is evaluated to the boolean `true`. No subsequent keys, values, or data types are permitted.

```
true
```

The document is evaluated to the string `"Hello, world"`. No subsequent keys, values, or data types are permitted.

```
"Hello, world"
```

The document is evaluated to the list `[1, 2, "three"]`. Note that there is no `::` indicator at the root as the vector value is directly at the root of the document without a key. No subsequent keys, values, or data types are permitted outside of the list elements. Since this is an inline list, no nesting is permitted either.

```
1, 2, "three"
```

The document is evaluated to the list `[1, 2, "three"]`. Note that there is no `::` indicator at the root as the vector value is directly at the root of the document without a key. No subsequent keys, values, or data types are permitted outside of the list elements.

```
- 1
- 2
- "three"
```

The document is evaluated to the dict `{"foo": 1, "bar": "two"}`. Note that there is no `::` indicator at the root as the vector value is directly at the root of the document without a parent key. Since this is an inline dict, no further nesting is permitted.

```
foo: 1, bar: "two"
```

The document is evaluated to the dict `{"foo": 1, "bar": "two", "baz": {"foo": "child"}}`. Note that there is no `::` indicator at the root as the vector value is directly at the root of the document without a key. Since the whole document in itself is now a dict, new keys and arbitrary nested types under them are permitted.

```
foo: 1
bar: "two"
baz::
 foo: "child"
```

The document is evaluated to the empty dict `{}` (special indicator). No subsequent keys, values, or data types are permitted.

```
{}
```

The document is evaluated to the empty list `[]` (special indicator). No subsequent keys, values, or data types are permitted.

```
[]
```

---

## Kitchensink example

```
This is the root dictionary for our application configuration.
application_config::
 application_name: "HUML Showcase Suite"
 version: "1.0.0-beta"
 environment: "development" # Can be 'production', 'staging', etc.
 debug_mode: true
 retry_attempts: 5
 timeout_seconds: 30.5
 feature_flags:: # A nested dictionary for feature toggles
 new_dashboard_enabled: true
 user_experiment_ab: false
 "legacy-system.compatibility_mode": true # Quoted key

Contact information
contact_points::
 - :: # List item: inline dictionary for admin
 type: "admin"
 email: "admin@example.com"
 phone: null # Null value example
 - :: # List item: multi-line dictionary for support
 type: "support"
 email: "support@example.com"
 availability::
 weekdays: "9am - 6pm"
 weekends: "10am - 2pm"

Numeric data types showcase
numerical_data::
 integer_val: 1_234_567
 float_val: -0.00789
 scientific_notation_val: 6.022e23
 hex_val: 0xCAFEBAFE
 octal_val: 0o755
 binary_val: 0b11011001
 infinity_positive: +inf
 infinity_negative: -inf
 not_a_number: nan
 empty_integer_list:: [] # Empty list
 empty_mapping:: {} # Empty dictionary

String variations
string_examples::
 simple_greeting: "Hello, \"Universe\"!"
 path_example: "C:\\Users\\Default\\Documents"
 multiline_preserved_poem: ```
 The HUML spec, so clear and bright,
 Makes data shine with pure delight.
 No ambiguity, no YAML fright,
 Just structured sense, and pure insight.
  ```
  multiline_stripped_script: """
    #!/bin/bash
    echo "Starting service..."
    # This script has leading spaces stripped.
    # Even this indented comment.
```

```

        exit 0
    """

# List variations
data_sources::
- "primary_db_connection_string"
- "secondary_api_endpoint_url"
- "192.168.1.100" # IP address as a string
- :: # A list of lists
  - "alpha"
  - "beta"
  - "gamma"
- true # A boolean in a list

inline_collections::
  simple_list:: "red", "green", "blue"
  simple_dict:: color: "yellow", intensity: 0.8, transparent: false
# List of inline dictionaries
points_of_interest::
  - :: x: 10, y: 20, label: "Start"
  - :: x: 15, y: 25, label: "Checkpoint 1"
  - :: x: 30, y: 10, label: "End"

# Example of a more complex nested structure
server_nodes::
- :: # First server node (dictionary)
  id: "node-alpha-001"
  ip_address: "10.0.0.1"
  roles:: "web", "api" # Inline list
  status: "active"
  "metadata with space": "custom server info" # Quoted key
  config_file_content: ```
    # Sample config for node-alpha-001
    port = 8080
    threads = 16
  ```
- :: # Second server node (dictionary)
 id: "node-beta-002"
 ip_address: "10.0.0.2"
 roles::
 - "database_primary"
 - "replication_master"
 status: "pending_maintenance"
 hardware_specs::
 cpu_cores: 8
 ram_gb: 64
 storage_tb: 2

Another top-level key, independent of 'application_config'
This demonstrates that a HUML file can have multiple top-level keys,
implicitly forming a root dictionary.
user_preferences::
 theme: "solarized_dark"
 font_size_pt: 12
 show_tooltips: true

```

[Kailash Nadh](#) / Website content is licensed under [CC BY-SA 4.0](#)